

Kriterion end-to-end review

18 September 2026 · read of the working copy as it stands uncommitted on your machine · 17,444 lines across 55 route, library and component files

Twenty-nine findings. Six of them put a wrong number or a false statement in front of a person today. The worst is the advisor PDF: for any submission whose advisory half is not finished, it prices the business off the wrong band table and prints a valuation roughly 40 percent of the correct figure, on a document headed with the client's company name.

The mess you are feeling is real and it has one cause. Nine screens and libraries each fetch the same rows and re-run the same engine with their own defaults. The 17 September work fixed that for the client report and the client PDF. It did not touch the other seven, and those seven are where every number below goes wrong.

How to read this

LIVE means a person can see the wrong thing right now. **LATENT** means the code is wrong but current data happens to hide it, and a named change would expose it. **TIDY** means nothing is wrong on screen, and the structure is what will keep producing the two categories above.

Every finding below I opened and read myself. Where I am relying on a reading I did not personally re-verify, I say so.

1. Numbers a person can see that are wrong

LIVE 1.1 The advisor PDF understates the valuation on every unreviewed submission

`src/lib/generate-submission-pdf.ts:262` · `src/lib/valscore_calc.js:195, 218`

The PDF computes `includeAdjusted` at line 112 and then never uses it for the valuation table. Line 262 reads `result.adjusted` unconditionally.

When no advisory answers exist, `valScore` equals the raw objective score out of 60, because `valScore = objectiveScore + advisoryScore` and the second term is zero. The adjusted leg then bands that raw number against `adjustedBands`, which run 0 to 100, and interpolates its multiple against `adjustedFloors`, which also run 0 to 100. A 45 out of 60 is read as a 45 out of 100.

The effect on the printed figure: the objective leg would give that submission roughly 1.17 times income. The adjusted leg gives roughly 0.45 times. On \$1.5M of net fee income the PDF prints about \$675,000 where the right answer is about \$1,755,000.

This is the same defect class as the 28 August PDF bug your notes record as fixed. It was fixed in the client report. It was never fixed here.

The PDF also prints an amber banner saying the advisory portion is pending, then prints the numbers anyway two inches below it.

LIVE 1.2 The admin results screen has the identical bug

`src/routes/admin.results.$submissionId.tsx:328, 505, 507`

Line 328 renders `<ValuationRow leg={result.adjusted} />` unconditionally. `advisoryComplete` is computed at line 220 and used only to show a caveat banner at line 293. It does not gate the row. Same wrong band, same understated valuation, on the screen an advisor is most likely to have open while talking to someone.

LIVE 1.3 The client report prices with EBITDA multiples and then labels the result Net Fee Income

`src/lib/client-report.ts:175` against `src/lib/valuation-defaults.ts:1`

Line 175 defaults a missing basis to `"ebitda"`. Every other site in the repo defaults to `"netfeeincome"`, including `/client` at `client.index.tsx:314` and `/client/assessment` at `client.assessment.tsx:179`, and that is what `DEFAULT_VALUATION_INPUT_TYPE` says.

The database permits a null type with a non-null amount. For such a row the report prices against the EBITDA anchors, which are roughly two to four times the net fee income anchors, and then line 311 labels the basis "Net Fee Income" because it tests for equality with `"ebitda"` and falls through. The same client sees a different valuation on `/client` than on `/client/summary`, and the summary's own label contradicts the summary's own arithmetic.

I have not confirmed that any current submission has a null type with a non-null amount. That is one query and worth running before you decide how urgent this is.

LIVE 1.4 The client score and the points available are on two different scales

`src/lib/client-report.ts:187` against 213 to 222

For an objective-only client the headline score at line 187 is `grossObjective(...)`, which puts a 45 out of 60 on screen as 75. The area totals at lines 213 to 220 are built from raw `max_score` and `actual_score`, so `totalAvailable` at line 222 stays in /60 units and reports 15.

The client therefore reads "75" next to "15 points still available across all eight areas" on `/client/summary` line 389 and in the PDF at `generate-client-pdf.ts:300`. Those add to 90. The comment above that block says the eight areas sum to 100, which is true for a reviewed submission and false for an objective-only one.

Line 350 of the summary compounds it by expressing the same raw number as a share of the grossed score: "N of the M points still available sit in this one area."

LIVE 1.5 Three screens round the same midpoint three different ways

`score-display.ts:127` defines `round10k`. `client.summary.tsx:336` uses it. `client.index.tsx:956` omits it. `advisor.plan.$submissionId.tsx:853` substitutes `Math.round`. `admin.results.$submissionId.tsx:59` and `:75` use a private `Intl` formatter.

Your 17 September rule was that the advisor must never see a different number from the client. The advisor plan screen, which is the screen where that conversation actually happens, is one of the four places still breaking it. It calls `formatCurrency(Math.round(...))` where the client calls `formatCurrency(round10k(...))`, so the two of you can be looking at midpoints up to \$5,000 apart.

The admin results screen is worse. It builds its own `fmtCurrency` with `Intl.NumberFormat` and prints \$1,646,667, the exact string the comment at `score-display.ts:137` exists to prevent, and its own `fmtRange` at line 75 uses `Math.round` rather than `round10k`.

Multiples split the same way. The client reads "1.17 times" and the advisor reads "1.2x", because `generate-submission-pdf.ts:23` and `admin.results:69` use `toFixed(1)` and everything client-facing uses `toFixed(2)`.

LIVE 1.6 \$1,500,000 can reach paper as a client's own income figure

`src/lib/valuation-defaults.ts:2`, used at `generate-submission-pdf.ts:81` and `admin.results.$submissionId.tsx:133`

`DEFAULT_VALUATION_INPUT_AMOUNT` is substituted when a submission has no income amount. On the advisor PDF it is then printed at line 246 under the heading "Financial Inputs, Amount", and a valuation derived from it is printed at line 269, on a document whose footer says "prepared for the named company". The admin results screen prints it at line 334 with no gate at all.

Meanwhile `advisor-queue.ts:461` tells the advisor that this same client cannot produce a valuation. Two of your own screens disagree about whether a number exists.

The advisor plan screen handles this correctly. It gates on `hasAmount` at lines 603 and 847 and shows nothing. That is the pattern the other two should copy.

LATENT 1.7 A score that falls in a gap between bands is labelled top band

`src/lib/score-display.ts:100`

`bandFor` matches on `score >= b.min && score <= b.max` against integer-bounded bands, and when nothing matches it falls back to `sorted.length - 1`, which is the highest band.

The bands are 0-49, 50-69, 70-84 and 85-100, so anything landing at 49.12 or 69.5 matches nothing and is labelled top band. `grossObjective` returns a float and `client-report.ts:195` passes it unrounded.

Today this does not fire, because with an objective maximum of exactly 60 the reachable grossed values all land on integers or outside the gaps. It fires the moment the objective maximum stops being 60. Your own conventions note that `/admin/questionnaire` recomputes `max_score` from the highest active option, which is precisely how the maximum would move without anyone intending it. At a maximum of 57, a score of 28 grosses to 49.12 and is shown to a client as 49 with the top band label beside it.

The same score reads "top band" on the client page and "—" on the admin page, because `valscore_calc.js:139` returns an empty string in the no-match case rather than guessing.

2. Words a person can see that are not true

LIVE 2.1 /client/assessment told objective-only clients an advisor was reviewing them

`src/routes/client.assessment.tsx`, five places

This is your open item 7 and it is fixed in this delivery. The route received an `awaitingReview` prop on 17 September and used it in exactly one place, the disclaimer at line 551. Every other sentence keyed off `reviewed` alone, which is false forever for an objective-only client, so they were told their advisor was working through the same eight areas independently and that the figures would move.

Detail of what changed is in section 6.

LIVE 2.2 /client/summary labels an objective-only score "Your ValScore"

`src/routes/client.summary.tsx:294`

The label is unconditional. For an objective-only client that number is `grossObjective(objectiveScore, objectiveMax)`, which `score-display.ts:52` warns in writing must not be conflated with a `ValScore`. The PDF gets this right and says "YOUR SCORE" at `generate-client-pdf.ts:176`. The screen and the PDF, which are supposed to be one report rendered twice, disagree on what the number is called.

LIVE 2.3 Unresolved template tags reach the client as literal braces

`src/lib/email-compose.functions.ts:303` · `src/lib/email-templates.ts:161, 230`

`{{top_area}}` and `{{top_points}}` are only populated inside an `if (opportunities.length > 0)` block, and `fillTags` deliberately leaves an unmatched tag as written. The shipped `review_ready` body contains the sentence "and `{{top_area}}` alone accounts for `{{top_points}}` of them, the largest single gap by a wide margin."

With zero opportunities, that is what the client receives. Nothing in the `blocked` or `warning` lists checks for unresolved tags, and the advisor reading the draft in the dialog would have to notice the braces themselves.

LIVE 2.4 The password reset template in /admin/emails will never reach anyone

`src/lib/email-templates.ts:106, 168` · `src/routes/client.auth.tsx:101`

Its own editor note claims it is "the only email on this list a client receives without you pressing anything". Nothing on that list is ever sent automatically. The reset email a client actually receives comes from Supabase Auth via `resetPasswordForEmail` at `client.auth.tsx:101`, using Supabase's default template and Supabase's sender, styled nothing like the rest.

So there is an editable, previewable, testable Kriterion-branded password reset email that no client will ever see, and an unbranded one they all will.

3. Access: who gets in, and where they land

LIVE 3.1 A retired invite code renders the full signup form

`src/routes/invite.tsx:97` · `src/lib/client-invites.functions.ts:232`

This is your open item 9, and the cause is that `/invite` has no loader and no `beforeLoad`. The only pre-flight check at line 97 tests whether the code string is empty. Validation happens inside the POST handler.

So someone with a retired link types their email, picks a password, confirms it, clicks "Create account & continue", waits through the spinner, and gets a transient toast. The form stays on screen filled in. A garbage code behaves identically, and the message does not distinguish a typo from a retired link.

Worth knowing alongside it: there is no "used" state. The `invite_codes` table has no `used_at`, no `redeemed_by`, no `max_uses`, and nothing writes back to it after a signup. Every active code is an unlimited-use public signup link. Forwarding one is all it takes.

LIVE 3.2 `/client/questionnaire` is missing the forced-password-change gate its three siblings have

`src/routes/client.questionnaire.tsx:8` to `16`

The `beforeLoad` checks session and client role and stops. `client.index.tsx:56`, `client.summary.tsx:50` and `client.assessment.tsx:60` all follow those two checks with a redirect to `/client/change-password` when `must_change_password` is set. A client bounced to change their password can type the questionnaire URL and start the assessment on their temporary password.

LIVE 3.3 An advisor can skip the forced password change entirely

`src/lib/require-advisor-auth.ts` · `src/lib/require-admin-auth.ts` · `src/routes/auth.tsx:117`

The `must_change_password` check exists in exactly one place on the advisor side, the sign-in handler at `auth.tsx:117`. Neither shared guard contains it. An advisor created with a temporary password who navigates straight to `/advisor` or `/admin/submissions` is in the back office with that temporary password still live, and `/advisor/change-password` is never enforced again.

LIVE 3.4 An "objective only" invite can email a full-service link

`src/lib/email-compose.functions.ts:229`

The line is `rows.find((c) => c.plan === wanted) ?? rows[0]`, where `rows` is every active code regardless of plan. If no active objective code exists, the composer silently picks whatever is first and builds the link from it. The advisor sees an opaque code in a URL and has no way to tell. The client signs up on the wrong plan and the plan is stamped onto their submission at registration.

The copy-a-link path does not have this fallback, so the two paths can disagree about what "objective" resolves to.

LIVE 3.5 The send handler enforces none of the reasons the draft handler computed for blocking

`src/lib/email-compose.functions.ts:343 to 411`

`getEmailDraft` works out five reasons a send should be blocked, including review not finished and zero advisory answers. `sendComposedEmail` re-derives none of them. Its validator checks string shapes. The only thing stopping a blocked send is a boolean in the browser at `send-email-dialog.tsx:116`.

The clearest consequence: `password_reset` is explicitly refused by the draft handler at line 319 and accepted by the send handler, which checks only that the key is one of four. The CTA URL is validated only against `/^https?:\/\//`. So a "someone asked to reset the password on this Kriterion account, the link works for one hour" email can be sent with an arbitrary link.

This requires an authenticated advisor. It is not an open door. It is a missing check on a path where the code already knows the answer.

LIVE 3.6 No email send is recorded anywhere

`src/lib/email-delivery.server.ts:90 · src/components/send-email-dialog.tsx:102, 109`

Success and failure both end as a toast. No log line, no database row, no retry, no webhook. There is no `send-log` table in the migrations. If the advisor navigates away or misses the toast, whether the email left is unknowable from inside the app, and nothing distinguishes "never attempted" from "Cloudflare refused" from "delivered" afterwards.

The one part of this that is done well: `email-delivery.server.ts:77` does not trust a 200 and checks the body for `success: true`, with the incident that prompted it written into the comment.

4. The structural cause

TIDY 4.1 Nine independent copies of "load this submission and compute it"

Every number in section 1 comes from the same shape of code. These nine sites each call `computeValuation` with their own defaults:

SITE	AMOUNT DEFAULT	BASIS DEFAULT
client-report.ts:171	0	ebitda
generate-submission-pdf.ts:99	1,500,000	netfeeincome
advisor-queue.ts:205	1,500,000	netfeeincome
email-compose.functions.ts:138	1,500,000	netfeeincome
admin.emails.tsx:345	1,500,000	netfeeincome
advisor.plan.\$submissionId.tsx:211	1,500,000	netfeeincome
client.index.tsx:310	0	netfeeincome
client.assessment.tsx:175	0	netfeeincome
admin.results.\$submissionId.tsx:144	1,500,000	netfeeincome

The 17 September work built `client-report.ts` so the client summary and the client PDF would stop drifting from each other. It worked. Those two agree. The other seven were not brought in, and they are where 1.1, 1.2, 1.3, 1.5 and 1.6 all live.

The "is this genuinely reviewed" rule is written out six separate times: `client-report.ts:162`, `client.index.tsx:106`, `advisor-queue.ts:193`, `client.assessment.tsx:214`, `email-compose.functions.ts:288` and `admin.emails.tsx:334`. The plan defaulting idiom `x === "objective" ? "objective" : "full"` appears four times in TypeScript and once more in SQL.

TIDY 4.2 Twelve hand-rolled route guards and no root layout

`__root.tsx` has no `beforeLoad` and there is no authenticated layout route. Every guard is per-route. There are two implementations of `requireAdvisorAuth`, the shared one and a local copy at `admin.clients.tsx:18`, and two of `requireAdminAuth`, the shared one and a local copy at `admin.advisors.tsx:23`. They disagree about where to send a rejected user, so a client who hits `/admin/clients` takes two redirects to land where one would do. The shared `requireAdminAuth` docstring describes a destination the function does not use.

Password length is specified three different ways: 6 at `auth.tsx:196`, 8 at `client.auth.tsx:157`, 8 at `invite.tsx:43`. Sign-in checks the client role before the advisor role while every guard checks advisor first, so a dual-role account always lands on the client side.

TIDY 4.3 Dead code that still looks live

- `/login` is a redirect to `/client/auth` with nothing linking to it, and the comment at `index.tsx:5` describes it as the team sign-in entry point, which it is not. The actual button at `index.tsx:156` points at `/client/auth`.
- `auth.tsx` carries a complete "create advisor account" signup branch at lines 124 to 135. `setMode` is never called, so none of it can run. The static copy two lines further down already says advisor accounts are created from the admin panel.
- `renderTargetBlock` in `generate-submission-pdf.ts:352` to 428 is an entire target-analysis renderer with its own money formatter, unreferenced. Its own comment says so.
- `inviteClient` and `createTestClient` in `client-invites.functions.ts` have no callers. `inviteClient` accepts a caller-supplied `redirectTo` validated only by `startsWith("http")`, which is an open redirect in an emailed link if anything ever wires it up.
- `public.section_scores` and `public.multiple_schedule` are still in the schema. Neither is read or written by any code. `multiple_schedule` is 23 columns of the original Excel sheet.
- `valscore_calc.js` is plain JavaScript with a hand-written `.d.ts`, and `tsconfig.json` has no `allowJs`, so the engine that produces every number in the product is the one file `tsc` never checks. The declarations happen to match the implementation today. Nothing enforces that.
- The `/admin/emails` preview does not use the renderer. Both file headers claim it does, specifically so that what an admin signs off is what leaves the building. The route imports one helper from `email-render.ts` and rebuilds the wordmark, button, link and footer in JSX, with the footer string duplicated.

5. The naming decision

You have three names in production at once. Here is every occurrence, so you can pick once and I can sweep it.

NAME	WHERE
Kriterion LLC	<code>index.tsx:9</code> page title, <code>:15</code> <code>og:title</code> , <code>:149</code> logo alt text, <code>:161</code> footer copyright · <code>__root.tsx:83</code> author meta
Kriterion BVI	<code>client.assessment.tsx</code> (three places, removed in this delivery) · <code>client.summary.tsx:305</code>
Kriterion alone	<code>email-render.ts:158</code> wordmark, <code>:164</code> and <code>:180</code> footer · <code>email-templates.ts</code> <code>fromName</code> in all four templates, plus subject and body copy at <code>:124</code> , <code>:127</code> , <code>:145</code> , <code>:172</code> , <code>:175</code> · <code>create_email_templates.sql:17</code> column default · <code>email-compose.functions.ts:377</code> fallback button label · <code>back-office-nav.tsx:144</code> · <code>client-shell.tsx:113</code> , <code>:162</code> · <code>client.summary.tsx:207</code> · <code>generate-client-pdf.ts:84</code> , <code>:387</code> · <code>generate-submission-pdf.ts:128</code> , <code>:326</code> · <code>invite.tsx:19</code> , <code>:64</code> , <code>:77</code> , <code>:89</code> · <code>__root.tsx:81</code> , <code>:84</code> , <code>:88</code>
Kriterion Business Value Intelligence	<code>email-render.ts:164</code> , <code>:180</code> · <code>admin.emails.tsx:812</code> · <code>__root.tsx:82</code> description

The domain is `kriterionbvi.com`, hardcoded as a fallback in three files: `email-compose.functions.ts:83`, `email-send.functions.ts:134` and `invite-client-button.tsx:33`. `PUBLIC_SITE_ORIGIN` is not in the Worker vars block in `vite.config.ts`, and that file's own comment records that a plain dashboard variable is wiped by the next push, so the hardcoded fallback is the operative value in production.

My read: the public marketing page is the only place saying LLC, and the email footer already carries the longer form. The cheapest consistent answer is Kriterion everywhere in product copy, Kriterion Business Value Intelligence in footers and the description meta, and the legal entity name only where a legal entity name belongs, which is a footer on the public page and nothing else. I have not changed any of it. Tell me which and I will sweep it in one pass.

6. What changed in this delivery

One file: `src/routes/client.assessment.tsx`. Open item 7, closed.

- The intro paragraph, the status badge, the "what happens next" card and the target planner's closing sentence now key off `awaitingReview` rather than `reviewed`, so an objective-only client is no longer told a review is under way.
- The objective-only client gets a new card, "Where this stands", saying their figures are final as they stand and that an advisor review is available as an add-on. The wording follows what `/client` already says at `client.index.tsx:565` and `:1070` rather than inventing a new promise.
- Two "Your Kriterion BVI advisor" sentences became "Contact Kriterion", because an objective-only client does not have an assigned advisor to contact. This also keeps the file out of the naming decision above.
- The file header now records the three states, so the next person does not write copy against `reviewed` alone again.

Gates: `bunx tsc --noEmit` exit 0. `bun run build` exit 0. `bunx eslint .` exit 0 errors, the 6 known shadcn fast-refresh warnings. No scoring code touched, no section maximum touched, no database change.

What I did not do: I have not seen this render. `/client/assessment` needs a client sign-in and I cannot sign in. You will need to check it as `andrewcdurden+test2@gmail.com` on an objective-only submission after you push.

7. What I did not check

- The live site. Everything here is a read of the code, not a walk of `kriterionbvi.com`.
- The database. I have not queried a single row, so every claim about what current data does is marked as such where it matters, particularly 1.3.
- The full-service invite link `KR-7F3A-9C21`, your open item 5. Still never walked end to end.
- The 24 buyer-consequence paragraphs Dan has not marked up. They are still live on the client result page.
- `QuestionnaireRunner.tsx`, `admin.questionnaire.tsx` and `admin.submissions.tsx` in any depth. That is 3,100 lines and the next largest unexamined surface.

8. What I would do next, in order

1. **1.1 and 1.2.** Gate the adjusted leg on `includeAdjusted` in the advisor PDF and on `advisoryComplete` on the admin results screen. Two small changes that stop a wrong valuation reaching paper. Half an hour.
2. **1.3.** Change one word at `client-report.ts:175` and run a query to find out whether any submission is affected. Ten minutes.

3. **1.5.** Delete the private formatters in admin.results and generate-submission-pdf, replace `Math.round` with `round10k` at advisor.plan:853, add `round10k` at client.index:956. An hour.
4. **3.1, 3.2, 3.3.** A `beforeLoad` on `/invite` that validates the code, and the missing `must_change_password` checks. An hour.
5. **1.4.** Decide what "points available" means for an objective-only client before changing anything. Grossing the available points to match the score is the obvious fix and it changes a number a client reads, so it is your call, not mine.
6. **4.1.** The rest of the report consolidation. This is the one that stops new versions of everything above. It is also the largest, and it should follow a preview rather than land as a diff.

Prepared 18 September 2026 from the uncommitted working copy. Line references are to that copy and will shift once anything is edited above them. Findings marked Live were each opened and read; the email subsystem findings in 2.3, 2.4, 3.4, 3.5 and 3.6 were located by a scan and I re-verified 3.4 and 3.5 directly.